Code Supplement: Implementation and User Guide for Auctions with Participant Audits

1 Overview

This supplement documents the code behind the numerical results: how the package is organized, how experiments are run, and what they output. The methods it implements (exact computation of the four mechanisms, the type distributions, the optimizers, uncertainty and checks) are described in the empirical supplement, Section 1. The code is the Python package `PA_auctions` in this repository; the user guide with every configuration key is the top-level `README.md`, and `PA_auctions/README.md` documents the methods at the level of functions.

2 Package Layout

`run.py` is the command-line entry point; `scripts/` holds the data scripts of the main paper and the empirical supplement, the notebooks that fetched and built the tweet data, descriptive statistics and the exploration of the raw Community Notes data, the previews of the synthetic families, and the accessibility check of figure colors; `scc/` holds the cluster scripts; `tests/` the test suite (`pytest`, 80 tests).

3 Running Experiments

A run is described by a TOML file and started with `python run.py CONFIG.toml`. Every list in the file is a set of values, and the run computes every combination; combinations that cannot run, such as the relaxation with $k \geq n$, are skipped and listed. There are two kinds of run.

- `optimize`: the sets are distributions (each with its externality scales ζ), markets, objectives, and optimizers. Each optimizer has a method (`genetic`, `best_response`, `genetic+best_response`, `relaxation`), its parameters, and its own list of threshold forms; a form the method cannot search is a configuration error. Given thresholds (below) may be added as benchmarks; each is evaluated in every market and reported as its own class in the gap decomposition.
- `evaluate`: the sets are distributions, ζ, markets, and given thresholds. A threshold is given as a polynomial, a constant, a tabulated function, a welfare cutoff (a number, or the ex ante cutoff), the threshold function of the symmetric relaxation, a saved `tau.json`, or every optimum of an earlier run.

Markets are either all pairs of the lists $n \in N$, $k \in K$, or an explicit list of pairs (n, k) (key `pairs`). The option `continuous` (default true) selects evaluation on continuous versions (empirical supplement, Section 1). Before running, the code prints the plan and a time estimate, calibrated by timing exact evaluations on each distribution. A search stopped by the time limit (default 60 minutes per combination) returns the best τ found so far, and the check `search_stop` fails for that combination. Combinations run in parallel worker processes (default: number of cores minus 2).

Module	Implements
`type_distributions.py`	type distributions as lists of cells (piecewise uniform, atomic), synthetic families, continuous and refined versions
`threshold_functions.py`	threshold functions $\tau(e)$ (picklable classes: polynomial, piecewise linear, cutoff, tabulated)
`threshold_forms.py`	parametric forms of τ searched by the optimizers (`poly<d>`, `knots<K>`, `bins<K>`, `cutoff`, `nodes`)
`scoring_rules.py`	score-based auctions: VCG, VCG-PA, IVCG-PA, first best
`exact_computation.py`	exact expected outcomes in quantile space: bid segments, ironing, interim allocations, Gauss–Legendre integration; continuous bid segments
`marginal_contribution.py`	marginal contributions M^τ, rival welfare loss L^τ, Pigouvian charges, participation gap
`genetic_search.py`	genetic search with Nelder–Mead refinement
`best_response_ascent.py`	best-response ascent, starts, line search, L-BFGS-B polish, per-iteration diagnostics
`symmetric_relaxation.py`	linear program of the symmetric relaxation and the threshold function read from it
`threshold_evaluation.py`	entry point: outcomes of the four mechanisms for a distribution and τ, exact and simulated
`auction_simulation.py`	simulated auctions on sampled type profiles (common random numbers)
`empirical.py`, `smoothing.py`	empirical populations: split, truncation, kernel smoothing, bootstrap
`pipeline.py`, `parallel.py`	building blocks of one combination; process-level parallelism within a combination
`figures.py`	all figures, drawn from saved files
`experiments/`	`config.py` (read, complete, validate and expand a TOML file), `runner.py` (plan, time estimate, parallel combinations), `compute.py` (one combination and its files), `report.py` (figures and README files), `provenance.py` (git, versions, host, seeds)

Table 1: Modules of `PA_auctions`.

Workers beyond one per combination run the parallel stages within a combination: search starts and restarts, Nelder–Mead starts, and the bootstrap. Table 2 lists the command-line options.

Reproducibility. The run record `run.toml` stores the following.

- The resolved configuration, with every default explicit.
- The random seeds: train/test split, search, simulation, bootstrap.
- The git commit, and whether tracked files had uncommitted changes. If they had, their diff is saved as `code.patch`.
- Package versions; the host (local machine, or the BU SCC cluster with its job id); the SHA-256 hash of each data file.
- Start and end times, and the runtime of each stage.

Rerunning the saved `config.toml` at the recorded commit reproduces the run. Figures are drawn only from saved files, so `python run.py --report RUN_FOLDER` redraws them without recomputation.

Test runs. Two configurations test the code.

Command	Does
`run.py CONFIG.toml`	plan, time estimate, confirmation, computation, figures, README files
`--dry-run`	plan and time estimate only
`--yes`	no confirmation (batch jobs)
`--workers N`	number of worker processes
`--no-report`	computation only, no figures
`run.py --report RUN`	figures and README files of an existing run, from its saved files
`--paper [NAMES]`	figures for a paper: one PDF per panel (section 4)
`run.py --options`	distributions, forms, objectives, optimizers, threshold sources and all defaults

Table 2: Command-line options of `run.py`.

- `configs/smoke_test.toml`: 6 combinations through every stage on coarse grids, about 15 seconds; for code changes.
- `configs/standard_test.toml`: every feature, about three minutes. It has two distributions: `harmful_middle`, and the tweet data at $\zeta \in \{5, 50\}$. It runs $n \in \{4, 8\}$, $k \in \{1, 2\}$, both objectives, and three optimizers: genetic search on `poly1`, `knots4` and `cutoff`, best-response ascent on `knots4`, and the relaxation on `nodes`, with short searches on coarse grids: 120 combinations.

Experiments of the main paper. The main paper and the empirical supplement use two runs, `configs/synthetic_run.toml` and `configs/shapes_run.toml`. With their run folders `SYN` and `SHP`,

```
python scripts/web27_paper_data.py SYN SHP OUT
python scripts/empirical_supplement_data.py SYN SHP OUT
```

write the tables, curves, type backgrounds and copied figures from which the two documents draw their figures in LaTeX.

BU SCC cluster. The same runs execute as batch jobs on the BU Shared Computing Cluster. The helper `scc/scc.sh`, run on the local machine, does the following.

- `submit CONFIG SLOTS HOURS`: push the branch, pull it on the cluster, print the plan, and submit the run with `SLOTS` worker processes and a run-time limit of `HOURS`. The cluster runs the last pushed commit.
- `status`, `log`: queue state; end of the newest job log.
- `fetch [RUN]`: copy run folders back, and merge the cluster's lines of `index.csv` into the local one.

Run folders on the cluster are on project storage; `run.toml` records the job id and slots. A single-node job takes 1–4, 8, 12, 16, 20, 28, 32 or 36 slots (other counts are rounded up); smaller requests start sooner.

4 Outputs

A run writes one folder, `<date>_<time>_<kind>_<name>`, with the files of table 3. Each combination has a sub-folder

`<distribution>/z<ζ>/n<n>-k<k>/<design>`.

The design label is `<objective>_<form>_<optimizer>` in optimize runs and `tau-<label>` in evaluate runs. A comparison figure goes in the `compare/` folder of the deepest level above every input it varies.

File	Contents
`README.md`	index: inputs, counts of check verdicts, results table with links to every figure, gap decomposition, figure reference
`config.toml`, `run.toml`	configuration as given; resolved configuration and provenance
`code.patch`, `log.txt`	uncommitted changes of tracked code (if any); progress messages
`summary.csv`	one row per combination: inputs, exact outcomes, gains, shares, revenue changes, bootstrap bounds, runtime
`checks.csv`	one row per check and combination
`gap-decomposition.csv`	the three gaps per distribution, ζ, market and class of thresholds; the relaxation value J and its gap to the best VCG-PA welfare found, with the distribution on which both are measured
`tau.json`, `tau.csv`	τ (rebuildable, reusable in evaluate runs); $\tau(e)$ and both penalty functions at the externality nodes
`results.json`	every statistic of a combination, including bootstrap intervals, search statistics, convergence trace, the relaxation record and the sufficient-condition measures
`first-order.csv`	per node and mechanism: threshold, X^τ, penalty, Pigouvian charge, marginal contribution, best-response gap, first-order violation
`data.npz`	arrays behind the figures: simulated outcomes, bootstrap resamples, ironing segments, maps of M^τ, tried thresholds
`types.npz`	per distribution and ζ: the type distribution drawn behind the figures
`README.md` (combination)	results table, files, each figure with its description and preview, figures not drawn and why
`runs/index.csv`	one line per run: folder, kind, start, runtime, combinations, failures, failed checks, commit, host

Table 3: Files of a run and of each combination.

Previews and README files. Every figure has a WebP preview at 120 pixels per inch in a `thumbs/` folder next to it. The README file of each combination and of each `compare/` folder shows the previews, each linked to its PDF and preceded by a description. Dense parts of the PDFs (scatter points, maps, tried thresholds) are embedded as images at 150 dpi; axes, text and curves stay vector.

Figures for a paper. The option `--paper` exports each panel of the selected figures as its own PDF at printed size (3.1×2.4 inches for panels of multi-panel figures, 3.9×2.9 for single panels), with fixed margins, so panels placed side by side align. Legends and colorbars go to their own PDFs. Each figure also gets a caption draft and a LaTeX `figure` environment that includes the panels. The run's own figures stay unchanged; the README files link the exported panels. The figures and tables of the main paper's empirical section are drawn in LaTeX from data written by `scripts/web27_paper_data.py`, and those of the empirical supplement from data written by `scripts/empirical_supplement_data.py`; both read the run folders of `configs/synthetic_run.toml` and `configs/shapes_run.toml`.

Figure	Shows	Drawn
1 types, τ, penalties	type distribution, τ, abstention region, excluded externalities, first-best boundary $v = -e$, bids pooled by IVCG-PA; penalty functions and recipient penalty $-e$	always
2 expected outcomes	expected welfare as bidder net utility + revenue + penalties + externality part, with the share of the first-best gain; changes over VCG of each quantity, with intervals	always
3 outcome distributions	per quantity: per-auction distribution of each mechanism, and of the paired difference from VCG (with the share of exact zeros); exact means and intervals marked	always
4 gain scatter	per auction: captured gain against achievable gain; revenue change against welfare change	always
5 ironing	$\hat{w}^\tau$ and $\bar{\omega}^\tau$ in quantile space, Ψ and its convex hull, their gap, scores against bids, interim allocations	ironing binds
6 marginal contribution	maps of $M^\tau(e, v)$ with the zero contour and τ; penalty against Pigouvian charge; M^τ at threshold types	always
search: tried	tried thresholds behind the best one	optimize
search: convergence	training objective against time, per start and stage	optimize
search: iterations	per iteration: share of the first-best gain, distance to the best welfare found, stationarity, first-order violation, best-response gap	optimize
compare: τ	τ of combinations that differ in one input (market, form, optimizer, objective, threshold); and τ by n at fixed k, by k at fixed n, and by k at fixed n/k	≥ 2 such
compare: ζ	welfare and gains over VCG against ζ	several ζ
compare: share	share of the first-best gain of every combination	always

Table 4: Figures. All are PDF. Each mechanism has a fixed color and line style, chosen to remain distinguishable under color-vision deficiencies.